PROYECTO DE DESARROLLO DE SOFTWARE

YACHANA DOMINÓ

Juego educativo de dominó para aprender lengua Inga

Documento técnico y guía de implementación
Frontend: HTML5 + CSS3 + JavaScript
Base de datos: SQL (PostgreSQL)

Basado en el documento de diseño Yachana Dominó, versión 1.0 (noviembre de 2026).

Propuesta de MVP académico e institucional
Colombia · 2026

Control del documento

Este documento transforma la propuesta funcional de Yachana Dominó en una guía inicial de desarrollo. Incluye arquitectura, módulos, estructura de carpetas, modelo relacional SQL y código base funcional para comenzar un prototipo web.

Alcance: se plantea una primera versión ejecutable en navegador. Las funciones que requieren servidor —cuentas Google reales, salas en línea, administración multiusuario y sincronización— se describen como siguientes etapas y no se simulan como si ya estuvieran implementadas.

Nota lingüística y cultural: las palabras Inga usadas como ejemplos proceden del documento de diseño. Deben ser revisadas y aprobadas por hablantes, docentes y autoridades de la comunidad antes de publicar el juego.

Elemento	Definición
Nombre del producto	Yachana Dominó
Propósito	Aprendizaje de vocabulario Inga mediante partidas de dominó.
Público	Estudiantes de 5 a 17 años, docentes e instituciones.
Tecnologías solicitadas	HTML, CSS, JavaScript y SQL.
Motor de base de datos propuesto	PostgreSQL.
Primera entrega (MVP)	Demo local de tablero, vocabulario inicial, reglas de juego y registro de resultados.

1. Resumen del proyecto

Yachana Dominó conserva la lógica del dominó tradicional, pero reemplaza los valores numéricos por conceptos de la lengua Inga. Cada mitad de ficha puede representar un dibujo, una palabra Inga, su traducción al español, números o audio. Dos mitades se conectan cuando representan el mismo concepto, aunque su formato visual sea diferente.

El juego busca apoyar el aprendizaje de vocabulario, pronunciación y cultura; ofrecer una herramienta para el aula; y permitir una futura oferta por licencias a instituciones.

Objetivos específicos

- Construir un tablero web adaptable a computador, tableta y teléfono.
- Permitir partidas de práctica con fichas y validación de conexiones por concepto.
- Mostrar vocabulario Inga-español y dejar preparado el soporte para imágenes y audios.
- Guardar usuarios, vocabulario, partidas y progreso en una base SQL.
- Preparar la evolución hacia perfiles, niveles, cursos y panel administrativo.

2. Alcance funcional del MVP

Módulo	Incluido en el prototipo inicial	Evolución posterior
Inicio	Pantalla de bienvenida y acceso a demo.	Inicio de sesión con Google verificado.
Tablero	Partida local, mano del jugador, cadena y pozo.	Rival inteligente y multijugador en línea.
Vocabulario	Conceptos iniciales, palabra Inga y español.	Audio nativo, imágenes y frases por nivel.
Progreso	Resultado básico y puntaje de partida.	Estrellas, monedas, rachas, estadísticas e insignias.
Administración	Carga inicial mediante SQL.	Panel web con roles, cursos, permisos y auditoría.

El documento de diseño propone seis niveles: Shuk, Ishkay, Kimsa, Chusku, Pichika y Sukta. Cada nivel contempla diez misiones y un desafío final. Para el MVP se recomienda iniciar con el nivel 1 y un conjunto pequeño de conceptos validados.

3. Reglas principales del juego

• Preparación: se seleccionan conceptos y se generan fichas con pares de conceptos.

• Reparto: el tamaño del conjunto puede ser mini (5 conceptos, 15 fichas), clásico (7 conceptos, 28 fichas) o grande (10 conceptos, 55 fichas).

• Turno: el jugador coloca una ficha si uno de sus conceptos coincide con un extremo de la cadena.

• Sin jugada: roba del pozo; si está vacío, pasa.

• Victoria: gana quien se queda sin fichas; si la partida se bloquea, gana quien tenga menos puntos en mano.

• Error: la ficha vuelve a la mano y se muestra la respuesta correcta; el error no debe quitar estrellas ni vidas.

Puntuación propuesta: unión correcta +10; respuesta en menos de cinco segundos +5; ficha doble +20. Los combos y recompensas adicionales se pueden incorporar después de validar el MVP.

4. Arquitectura propuesta

Para cumplir el requisito HTML + CSS + JS + SQL, se separa la interfaz del almacenamiento. HTML presenta la estructura; CSS define la identidad visual; JavaScript administra turnos y reglas; PostgreSQL almacena usuarios, vocabulario y resultados. En una instalación real, el navegador no debe conectarse directamente a PostgreSQL: se necesita una API de servidor que autentique y valide las operaciones.

Capa	Tecnología	Responsabilidad
Interfaz	HTML5	Pantallas, tablero, fichas, controles y accesibilidad.
Estilos	CSS3	Diseño adaptable, paleta, estados, animaciones.
Lógica cliente	JavaScript	Generación de fichas, turnos, conexiones, puntaje.
API futura	Node.js / NestJS	Autenticación, reglas autoritativas y acceso seguro a datos.
Datos	PostgreSQL / SQL	Usuarios, perfiles, vocabulario, partidas y progreso.

Flujo de una jugada: estudiante toca ficha → JavaScript verifica coincidencia → tablero actualiza la cadena → se calcula puntaje → cuando exista API, se envía el evento validado al servidor → SQL registra el resultado.

5. Estructura de carpetas

Estructura sugerida del proyecto

```
yachana-domino/
├── index.html
├── css/
│   └── styles.css
├── js/
│   ├── app.js
│   ├── game.js
│   └── vocabulary.js
├── assets/
│   ├── images/
│   └── audio/
├── database/
│   └── schema.sql
└── README.md
```

Para probar el prototipo local, abra la carpeta en Visual Studio Code y ejecute index.html con Live Server. La persistencia local del ejemplo se mantiene durante la partida; el guardado permanente se incorpora al conectar una API y PostgreSQL.

6. Desarrollo del frontend

6.1 index.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Yachana Dominó</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <header class="topbar">
    <div>
      <p class="eyebrow">Une, juega y aprende Inga</p>
      <h1>Yachana <span>Dominó</span></h1>
    </div>
    <div class="score">
      <span>Nivel 1 · Shuk</span>
      <strong id="score">0 puntos</strong>
    </div>
  </header>

  <main class="layout">
    <section class="intro card">
      <div>
        <h2>¡Aprendamos jugando!</h2>
        <p>Une conceptos iguales. La palabra puede aparecer en Inga,
           en español o como dibujo.</p>
      </div>
      <button id="newGame" class="button secondary">Nueva partida</button>
    </section>

    <section class="board card" aria-label="Tablero de dominó">
      <div class="section-heading">
        <h2>Mesa de juego</h2>
        <span id="status" role="status">Tu turno</span>
      </div>
      <div id="chain" class="chain">
        <p class="empty">Coloca la primera ficha para comenzar.</p>
      </div>
    </section>

    <section class="hand card">
      <div class="section-heading">
        <h2>Tus fichas</h2>
        <span id="handCount">0 fichas</span>
      </div>
      <div id="hand" class="tiles"></div>
      <button id="draw" class="button">Robar del pozo</button>
    </section>

    <aside class="learn card">
      <h2>Palabra para aprender</h2>
      <p id="learnWord" class="word">yaku</p>
      <p id="learnTranslation">agua</p>
      <p class="hint">El vocabulario es una base de trabajo que requiere validación comunitaria.</p>
    </aside>
  </main>

  <footer>Yachana Dominó · Proyecto educativo intercultural</footer>
  <script src="js/vocabulary.js"></script>
  <script src="js/game.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

6.2 css/styles.css

```
:root {
  --wine: #7A1F3D;
  --sun: #FFC629;
  --turquoise: #12B5CB;
  --pink: #E5338A;
  --green: #35B86B;
  --orange: #FF7B1C;
  --ink: #302832;
  --paper: #FFF9F1;
  --white: #FFFFFF;
}
* { box-sizing: border-box; }
body {
  margin: 0; color: var(--ink); background: var(--paper);
  font-family: "Nunito", "Trebuchet MS", sans-serif;
}
.topbar {
  display: flex; justify-content: space-between; align-items: center;
  gap: 16px; padding: 20px clamp(16px, 5vw, 64px);
  color: white; background: var(--wine);
}
h1 { margin: 0; font-size: clamp(1.7rem, 4vw, 2.6rem); }
h1 span { color: var(--sun); }
.eyebrow { margin: 0 0 5px; letter-spacing: .08em; }
```

```
.score { text-align: right; display: grid; gap: 5px; }
.layout {
  width: min(1180px, 94%); margin: 24px auto;
  display: grid; grid-template-columns: 1.4fr 1fr;
  gap: 18px;
}
.card {
  background: var(--white); border-radius: 18px; padding: 20px;
  box-shadow: 0 8px 24px #35132212;
}
.intro { grid-column: 1 / -1; display:flex; justify-content:space-between;
  align-items:center; gap:18px; border-left: 7px solid var(--sun); }
h2 { margin: 0 0 10px; color: var(--wine); }
.section-heading { display:flex; justify-content:space-between; gap:10px; }
#status { color: var(--green); font-weight: 800; }
.chain, .tiles { display:flex; flex-wrap:wrap; gap:10px; min-height:90px;
  align-items:center; padding:12px 0; }
.tile {
  display:grid; grid-template-columns:1fr 1fr; width:136px; min-height:70px;
  border:2px solid var(--wine); border-radius:12px; overflow:hidden;
  background:#fff; cursor:pointer; padding:0;
}
.tile:focus-visible, button:focus-visible {
  outline:3px solid var(--turquoise); outline-offset:3px;
}
.half { padding:10px 5px; text-align:center; font-weight:800; }
.half + .half { border-left:1px solid #d8cbd0; background:#fff6e4; }
.concept { display:block; color:var(--wine); }
.translation { display:block; font-size:.72rem; color:#655b61; }
.button {
  border:0; border-radius:999px; padding:12px 20px;
  background:var(--orange); color:white; font-weight:800; cursor:pointer;
}
.button.secondary { background:var(--turquoise); color:#12343a; }
.word { font-size:2rem; color:var(--wine); font-weight:900; margin-bottom:0; }
.hint, footer { color:#71666d; font-size:.82rem; }
footer { text-align:center; padding:20px; }
.empty { color:#766c72; }
@media (max-width:720px) {
  .layout { grid-template-columns:1fr; }
  .intro { grid-column:auto; align-items:flex-start; flex-direction:column; }
  .topbar { align-items:flex-start; flex-direction:column; }
  .score { text-align:left; }
}
```

6.3 js/vocabulary.js

```
// Vocabulario demostrativo tomado de la propuesta de diseño.
// Confirmar escritura, significado y pronunciación con hablantes Inga.
const VOCABULARY = [
  { id: "shuk", inga: "shuk", es: "uno", category: "números" },
  { id: "ishkay", inga: "ishkay", es: "dos", category: "números" },
  { id: "kimsa", inga: "kimsa", es: "tres", category: "números" },
  { id: "inti", inga: "inti", es: "sol", category: "naturaleza" },
  { id: "yaku", inga: "yaku", es: "agua", category: "naturaleza" },
  { id: "urku", inga: "urku", es: "montaña", category: "naturaleza" }
];
```

6.4 js/game.js

```
function createDeck(concepts) {
  const deck = [];
  for (let i = 0; i < concepts.length; i++) {
    for (let j = i; j < concepts.length; j++) {
      deck.push({ id: crypto.randomUUID(),
        left: concepts[i], right: concepts[j] });
    }
  }
  return deck;
}

function shuffle(items) {
  const copy = [...items];
  for (let i = copy.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [copy[i], copy[j]] = [copy[j], copy[i]];
  }
  return copy;
}

function canConnect(tile, endpoint) {
  return tile.left.id === endpoint.id || tile.right.id === endpoint.id;
}

function orientTile(tile, endpoint) {
  if (tile.left.id === endpoint.id) return tile;
  return { ...tile, left: tile.right, right: tile.left };
}
```

6.5 js/app.js

```
let deck = [];
let hand = [];
let chain = [];
let points = 0;
```

```
const $ = (id) => document.getElementById(id);

function startGame() {
  deck = shuffle(createDeck(VOCABULARY));
  hand = deck.splice(0, 5);
  chain = [];
  points = 0;
  render();
  $("status").textContent = "Tu turno";
}

function renderTile(tile, onClick, disabled = false) {
  const button = document.createElement("button");
  button.className = "tile";
  button.type = "button";
  button.disabled = disabled;
  button.setAttribute("aria-label",
    `${tile.left.inga} con ${tile.right.inga}`);
  [tile.left, tile.right].forEach((concept) => {
    const half = document.createElement("span");
    half.className = "half";
    const name = document.createElement("span");
    name.className = "concept";
    name.textContent = concept.inga;
    const translation = document.createElement("span");
    translation.className = "translation";
    translation.textContent = concept.es;
    half.append(name, translation);
    button.append(half);
  });
  if (onClick) button.addEventListener("click", onClick);
  return button;
}

function playTile(index) {
  const tile = hand[index];
  if (!tile) return;

  if (chain.length === 0) {
    chain.push(tile);
  } else {
    const leftEnd = chain[0].left;
    const rightEnd = chain[chain.length - 1].right;
    if (canConnect(tile, rightEnd)) {
      chain.push(orientTile(tile, rightEnd));
    } else if (canConnect(tile, leftEnd)) {
      const oriented = orientTile({left:tile.right, right:tile.left}, leftEnd);
      chain.unshift(oriented);
    } else {
      $("status").textContent = "No coincide. Intenta otra ficha.";
      $("learnWord").textContent = tile.left.inga;
      $("learnTranslation").textContent = tile.left.es;
      return;
    }
  }
  hand.splice(index, 1);
  points += 10;
  $("status").textContent = hand.length === 0 ? "¡Ganaste!" : "¡Buena jugada!";
  render();
}

function render() {
  $("score").textContent = `${points} puntos`;
  $("handCount").textContent = `${hand.length} fichas`;
  $("hand").replaceChildren();
  $("chain").replaceChildren();

  hand.forEach((tile, index) =>
    $("hand").append(renderTile(tile, () => playTile(index))));
  if (chain.length === 0) {
    const empty = document.createElement("p");
    empty.className = "empty";
    empty.textContent = "Coloca la primera ficha para comenzar.";
    $("chain").append(empty);
  } else {
    chain.forEach(tile => $("chain").append(renderTile(tile, null, true)));
  }
}

$("newGame").addEventListener("click", startGame);
$("draw").addEventListener("click", () => {
  if (deck.length && hand.length < 10) hand.push(deck.pop());
  $("status").textContent = deck.length ? "Ficha tomada del pozo." : "Pozo vacío.";
  render();
});
startGame();
```

Este código es una base didáctica. Para una versión de producción se debe ampliar el control de extremos, la lógica de robo y pase, el reparto entre jugadores, la condición de bloqueo, el modo de rival y la validación de jugadas en servidor. El prototipo no sustituye una prueba completa del juego.

7. Base de datos SQL

El siguiente esquema usa PostgreSQL. Incluye las entidades principales descritas en el documento funcional: usuarios, perfiles, reglas de acceso, instituciones, cursos, vocabulario, niveles, misiones, partidas, progreso y auditoría. Los campos de archivos guardan URL; los audios e imágenes se almacenan en un servicio de archivos, no como binarios dentro de estas tablas.

7.1 database/schema.sql

```
CREATE TABLE institutions (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(160) NOT NULL,
  license_plan VARCHAR(30) DEFAULT 'demo',
  license_expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE app_users (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  google_sub VARCHAR(255) UNIQUE,
  email VARCHAR(254) NOT NULL UNIQUE,
  role VARCHAR(30) NOT NULL DEFAULT 'student'
    CHECK (role IN ('student','teacher','institution_admin','superadmin')),
  status VARCHAR(20) NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','suspended','blocked','pending_deletion','deleted')),
  status_reason TEXT,
  suspended_until TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);

CREATE TABLE classes (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  institution_id BIGINT REFERENCES institutions(id),
  name VARCHAR(100) NOT NULL,
  classroom_code VARCHAR(20) UNIQUE,
  teacher_id BIGINT REFERENCES app_users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE profiles (
  user_id BIGINT PRIMARY KEY REFERENCES app_users(id) ON DELETE CASCADE,
  nickname VARCHAR(40) NOT NULL,
  avatar VARCHAR(80) DEFAULT 'kuyi',
  age_range VARCHAR(10),
  institution_id BIGINT REFERENCES institutions(id),
  class_id BIGINT REFERENCES classes(id),
  interface_language VARCHAR(10) DEFAULT 'es',
  ranking_visible BOOLEAN DEFAULT FALSE,
  sara_coins INTEGER NOT NULL DEFAULT 0 CHECK (sara_coins >= 0)
);

CREATE TABLE access_rules (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  rule_type VARCHAR(20) NOT NULL
    CHECK (rule_type IN ('domain','allow_email','deny_email')),
  rule_value VARCHAR(254) NOT NULL,
  institution_id BIGINT REFERENCES institutions(id),
  created_by BIGINT REFERENCES app_users(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(rule_type, rule_value, institution_id)
);

CREATE TABLE vocabulary (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  inga_word VARCHAR(120) NOT NULL,
  spanish_translation VARCHAR(160) NOT NULL,
  image_url TEXT,
  audio_url TEXT,
  category VARCHAR(80) NOT NULL,
  level_number SMALLINT NOT NULL CHECK (level_number BETWEEN 1 AND 6),
  validation_status VARCHAR(20) NOT NULL DEFAULT 'pending'
    CHECK (validation_status IN ('pending','validated','rejected')),
  validated_by BIGINT REFERENCES app_users(id),
  validated_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE levels (
  id SMALLINT PRIMARY KEY CHECK (id BETWEEN 1 AND 6),
  inga_name VARCHAR(80) NOT NULL,
  theme VARCHAR(120) NOT NULL,
  age_min SMALLINT,
  age_max SMALLINT,
  unlock_stars SMALLINT DEFAULT 0
);

CREATE TABLE missions (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  level_id SMALLINT NOT NULL REFERENCES levels(id),
  mission_number SMALLINT NOT NULL CHECK (mission_number BETWEEN 1 AND 11),
  title VARCHAR(120) NOT NULL,
  rules JSONB NOT NULL DEFAULT '{}'::jsonb,
  UNIQUE(level_id, mission_number)
);
```

```
CREATE TABLE matches (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  mode VARCHAR(30) NOT NULL,
  level_id SMALLINT REFERENCES levels(id),
  score INTEGER NOT NULL DEFAULT 0,
  accuracy NUMERIC(5,2),
  duration_seconds INTEGER,
  result VARCHAR(20),
  played_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE match_players (
  match_id BIGINT REFERENCES matches(id) ON DELETE CASCADE,
  user_id BIGINT REFERENCES app_users(id) ON DELETE CASCADE,
  final_score INTEGER DEFAULT 0,
  outcome VARCHAR(20),
  PRIMARY KEY(match_id, user_id)
);

CREATE TABLE progress (
  user_id BIGINT REFERENCES app_users(id) ON DELETE CASCADE,
  mission_id BIGINT REFERENCES missions(id) ON DELETE CASCADE,
  stars SMALLINT NOT NULL DEFAULT 0 CHECK (stars BETWEEN 0 AND 3),
  completed_at TIMESTAMPTZ,
  PRIMARY KEY(user_id, mission_id)
);

CREATE TABLE word_stats (
  user_id BIGINT REFERENCES app_users(id) ON DELETE CASCADE,
  vocabulary_id BIGINT REFERENCES vocabulary(id) ON DELETE CASCADE,
  correct_count INTEGER NOT NULL DEFAULT 0,
  error_count INTEGER NOT NULL DEFAULT 0,
  next_review_at TIMESTAMPTZ,
  PRIMARY KEY(user_id, vocabulary_id)
);

CREATE TABLE audit_logs (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  actor_id BIGINT REFERENCES app_users(id),
  action VARCHAR(80) NOT NULL,
  target_type VARCHAR(50),
  target_id VARCHAR(100),
  reason TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_vocabulary_level ON vocabulary(level_number);
CREATE INDEX idx_users_status ON app_users(status);
CREATE INDEX idx_matches_played_at ON matches(played_at);
CREATE INDEX idx_audit_created_at ON audit_logs(created_at);
```

7.2 Datos iniciales de demostración

```
INSERT INTO levels (id, inga_name, theme, age_min, age_max)
VALUES
(1, 'Shuk - Sara', 'Números y colores', 5, 7),
(2, 'Ishkay - Kuy', 'Animales y familia', 6, 9),
(3, 'Kimsa - Yaku', 'Naturaleza y alimentos', 8, 11),
(4, 'Chusku - Urku', 'Saludos y frases', 10, 13),
(5, 'Pichika - Inti', 'Oraciones y números', 12, 15),
(6, 'Sukta - Kuntur', 'Cultura y relatos', 14, 17);

INSERT INTO vocabulary
(inga_word, spanish_translation, category, level_number, validation_status)
VALUES
('shuk', 'uno', 'números', 1, 'pending'),
('ishkay', 'dos', 'números', 1, 'pending'),
('kimsa', 'tres', 'números', 1, 'pending'),
('puka', 'rojo', 'colores', 1, 'pending'),
('yuraj', 'blanco', 'colores', 1, 'pending'),
('yana', 'negro', 'colores', 1, 'pending'),
('killu', 'amarillo', 'colores', 1, 'pending');
```

Los registros se dejan en estado pending deliberadamente. Un administrador autorizado debe cambiar el estado a validated únicamente después de la revisión lingüística comunitaria.

8. Diseño de seguridad y privacidad

• Nunca guardar contraseñas de Google. La autenticación real debe validar el token en el servidor y crear una sesión segura.

• No conectar el navegador directamente a PostgreSQL ni incluir credenciales de base de datos en JavaScript.

• Aplicar control de acceso por roles: estudiante, docente, administrador de institución y superadministrador.

• Implementar listas blancas, listas negras y dominios autorizados desde la API.

• Registrar suspensiones, bloqueos y eliminaciones con motivo obligatorio y auditoría no editable.

• Recolectar datos mínimos de estudiantes; evitar dirección, teléfono, fotografías y chat libre.

• Solicitar autorización del acudiente cuando corresponda y revisar las obligaciones legales colombianas antes del despliegue.

La propuesta original menciona la Ley 1581 de 2012 y sus decretos reglamentarios como marco de protección de datos. Este documento es una guía técnica y no reemplaza asesoría jurídica ni una política formal de tratamiento de datos.

9. Accesibilidad, identidad visual y contenido

La identidad visual propuesta usa guinda #7A1F3D, amarillo sol #FFC629, turquesa #12B5CB, magenta #E5338A, verde #35B86B, naranja #FF7B1C, azul #3A63E0 y morado #7B4FD0. Se recomiendan botones grandes, formas redondeadas, texto legible y animaciones suaves.

Personajes propuestos: Kuyi, cuy explorador para niveles iniciales, y Kuntur, cóndor sabio para niveles avanzados. El arte, los patrones, los sonidos y los relatos deben construirse y revisarse con la comunidad, evitando representar símbolos culturales sin autorización.

Accesibilidad prevista: alto contraste, opción para daltonismo, tipografía amigable para dislexia, texto grande, lectura en voz alta, control por teclado, reducción de movimiento y controles separados de música, efectos y voz.

10. Pruebas recomendadas

Prueba	Resultado esperado
Conectar dos mitades del mismo concepto	La ficha se acepta aunque una mitad sea palabra y la otra imagen/audio.
Intentar conectar conceptos diferentes	La ficha no se agrega; se muestra una explicación amable.
Robar con pozo disponible	La ficha pasa a la mano y disminuye el pozo.
Robar con pozo vacío	El sistema informa que no quedan fichas.
Nueva partida	Se reinician cadena, mano, pozo y puntaje.
Diseño móvil	Botones y fichas se pueden usar sin zoom horizontal.
Vocabulario pendiente	No se publica como contenido oficial validado.
Acceso bloqueado	La API deniega la sesión y registra el evento.

11. Ruta de desarrollo

Fase	Entregables	Estimación del diseño b
0. Contenido y validación	Banco de 150–200 palabras con audio e ilustración; arte base.	3–4 semanas
1. MVP	Ingreso, perfil, niveles 1–3, tablero, demo y panel básico.	6–8 semanas
2. Aula	Niveles 4–6, cursos, códigos, torneos, reportes y pronunciación.	5–7 semanas
3. Premium	PWA sin conexión, tienda, familias, certificados y marca institucional.	4–6 semanas
4. Piloto	Pruebas con 2–3 instituciones, ajustes y capacitación.	4 semanas

Los tiempos son estimaciones de la propuesta de diseño y pueden variar según el tamaño del equipo, la disponibilidad de vocabulario validado, audios, ilustraciones y conectividad.

12. Cómo ejecutar el prototipo

• Crear la carpeta y los archivos con la estructura indicada.

• Copiar cada bloque de código en su archivo correspondiente.

• Abrir index.html con Live Server desde Visual Studio Code.

• Probar nueva partida, selección de fichas, conexiones correctas e incorrectas y botón de robo.

• Instalar PostgreSQL y ejecutar database/schema.sql desde pgAdmin o psql para crear la estructura de datos.

• Para guardar resultados desde el navegador, desarrollar una API Node.js que reciba las jugadas y consulte PostgreSQL de forma segura.

Importante: el frontend de este documento es un prototipo demostrativo; el SQL es el esquema inicial de persistencia. No se debe presentar como una plataforma completa con autenticación real, multijugador, panel administrativo o protección de producción hasta desarrollar y probar la API, controles de seguridad y procesos de despliegue.

13. Conclusión

Yachana Dominó puede desarrollarse por etapas: primero un tablero web sencillo que permita jugar y aprender conceptos; después perfiles y progreso persistente; y finalmente cursos, torneos, panel administrativo, audio, modo sin conexión y licenciamiento institucional. La validación lingüística y cultural debe acompañar todas las etapas y no dejarse para el final.

El siguiente paso técnico recomendado es implementar el MVP con el nivel 1, vocabulario validado, una partida local completa y una API básica para guardar resultados.

Anexo. Referencia del documento fuente

Este proyecto toma como base el archivo Yachana_Domino_Inga_Documento_de_Diseno.pdf, versión 1.0, noviembre de 2026. Se incorporaron especialmente: visión y objetivos (pág. 2), identidad visual (pág. 3), mecánica (pág. 4), niveles (págs. 5–6), modos y vocabulario (pág. 7), demos y registro (págs. 8–9), perfil y administración (págs. 10–12), funciones premium (pág. 13), pantallas (pág. 14), arquitectura y modelo de datos (pág. 15), seguridad y accesibilidad (pág. 16), y hoja de ruta (pág. 17).

Documento elaborado como especificación y base de código inicial. Versión técnica: 1.0 · Noviembre de 2026.